

CURSO 3: CREACIÓN Y DESARROLLO DE AGENTES DE IA

CLASE 07 • NIVEL 3 - AVANZADO

Clase 07: Agentes Autónomos y el Ciclo Cognitivo ReAct

«El Agente como un Detective que Piensa, Actúa y Observa hasta Resolver el Caso»

Guía de estudio oficial y manual técnico. Diseñado para dominar la programación en Python mediante modelos mentales rigurosos, diagramas de flujo y código de producción.

Autor & Mentor: William Rodríguez (Wisrovi)

Rol: AI Solutions Architect & Principal Software Engineer

Python: 3.10+ | **Licencia:** MIT | **wisrovi SUITE**

Acerca del Autor y Mentor



William Rodríguez (Wisrovi)

AI Solutions Architect & Principal Software Engineer • Badajoz, España

Ingeniero y arquitecto de software especializado en Inteligencia Artificial Generativa, sistemas multi-agente, Visión por Computador e infraestructuras MLOps de alta disponibilidad. Creador y mantenedor de la suite de software libre wisrovi SUITE en PyPI con más de 26 bibliotecas enfocadas en orquestación de pipelines, caching distribuido y optimización de bases de datos.



GitHub: github.com/wisrovi



LinkedIn: [in/wisrovi-rodriguez](https://in.linkedin.com/in/wisrovi-rodriguez)



DockerHub: hub.docker.com/u/wisrovi



Website: wisrovi.dev

Metodología de Aprendizaje: La Regla de la Bicicleta

En este programa no creemos en el aprendizaje pasivo. Programar no se aprende memorizando manuales o mirando videos en segundo plano mientras tomas café; se aprende **escribiendo código**, enfrentando errores de sintaxis, depurando variables y viendo cómo responde el intérprete en tiempo real.



El Compromiso Activo del Estudiante

Abre Visual Studio Code en cada sesión. Escribe cada ejemplo con tus propias manos. Cambia los números, rompe el código deliberadamente para ver el mensaje de error de Python, y luego arréglalo. Ese proceso de experimentación es el que construye sinapsis duraderas.

Presentación de la Sesión

Bienvenido/a a la **CLASE 07** del **Curso 3: Creación y Desarrollo de Agentes de IA**. Esta guía técnica está estructurada para proporcionarte las bases conceptuales, arquitectónicas y prácticas indispensables para tu progreso.

🎯 Objetivos de la Sesión

Competencia Conceptual: Comprender el paradigma ReAct (Reasoning + Acting), bucles de decisión autónoma y memoria de trabajo.

Competencia Práctica: Construir un agente autónomo capaz de resolver tareas multi-paso llamando herramientas en bucle.

Estructura del Documento

Sección	Contenido Temático	Objetivo Pedagógico
Pág 4: Fundamento	Teoría, Modelo Mental y Metáfora	Comprensión del concepto
Pág 5: Flujo	Diagrama de Arquitectura de Memoria	Visualización del flujo interno
Pág 6: Código	Implementación en Python 3.10+	Patrones idiomáticos (PEP 8)
Pág 7: Gotchas	Antipatrones y Trampas Comunes	Prevención de bugs en producción
Pág 8: Conclusiones	Cierre de Sesión & Notas de Repaso	Consolidación del aprendizaje
Pág 9: Bibliografía	Fuentes Canónicas y Desafío	Ampliación y autoestudio

Clase 07: Agentes Autónomos y el Ciclo Cognitivo ReAct

Un Agente de IA es un sistema autónomo que combina un LLM con memoria, herramientas y un bucle de razonamiento para alcanzar metas.

☀ **Metáfora Central: El Agente como un Detective que Piensa, Actúa y Observa hasta Resolver el Caso**

Un agente es como un detective: piensa qué pista necesita (Thought), busca el dato con una herramienta (Action), analiza el resultado (Observation) y repite.

Principios Teóricos y Modelo Mental

Ciclo ReAct: Thought (Pensamiento) -> Action (Acción/Tool) -> Observation (Resultado) -> Final Answer.

El agente evalúa si la observación resuelve el objetivo o si requiere ejecutar acciones adicionales.

⚡ **Regla de Oro en Python**

Establece siempre un 'max_iterations = 5' para evitar que el agente quede atrapado en bucles infinitos.

Arquitectura y Movimiento de Datos en Memoria

Bucle autónomo de decisión ReAct con parada por condición de meta.



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Intérprete	Estado en Memoria
1. Inicialización	Recepción de la meta del usuario.	Objetivo fijado en memoria de trabajo.
2. Evaluación	Generación de Thought y selección de Action.	Herramienta elegida con parámetros.
3. Transformación	Ejecución de la herramienta y captura de Observation.	Nuevo dato agregado al historial.
4. Retorno / Salida	¿Meta alcanzada? Sí -> Final Answer / No -> Siguiente ciclo.	Respuesta final entregada.



Visualización Mental

El bucle ReAct le da al modelo tiempo para pensar y verificar antes de responder.

Código de Demostración en Python 3.10+

Agente ReAct autónomo con límite de pasos implementado en Python:

main.py (Python 3.10+)

PEP 8 Compliant

```
class ReActAgent:
    def __init__(self, tools: dict, max_steps: int = 3):
        self.tools = tools
        self.max_steps = max_steps
        self.memory = []

    def run(self, goal: str):
        print(f"🎯 Meta: {goal}")
        for step in range(1, self.max_steps + 1):
            print(f"--- Paso {step} ---")
            # 1. Thought
            thought = f"Necesito consultar la cotización del euro."
            print(f"💭 Thought: {thought}")

            # 2. Action
            obs = self.tools["get_rate"]("EUR_USD")
            print(f"🛠️ Action: get_rate(EUR_USD) -> Obs: {obs}")

            # 3. Final Answer
            return f"Respuesta Final: 1 EUR equivale a {obs} USD."

tools = {"get_rate": lambda pair: 1.08}
agente = ReActAgent(tools)
print(agente.run("¿Cuánto vale el euro frente al dólar?"))
```

Análisis de Ingeniería del Código

Arquitectura de agente con inyección de herramientas, bucle de ejecución y memoria de observaciones.

💡 Buena Práctica de Tipado

El uso sistemático de Type Hints (PEP 484) permite a los editores modernos como VS Code activar autocompletado inteligente y detectar errores antes de la ejecución.

Trampas Frecuentes de Depuración (Gotchas)

Bucles infinitos en agentes autónomos.

⚠ Gotcha Frecuente (Trampa de Principiante)

Si una herramienta falla o devuelve error, el agente puede reintentar la misma acción en un ciclo sin fin.

Comparativa: Antipatrón vs Patrón Recomendado

❌ Antipatrón

```
while not finished: agent.step() # ❌ Puede  
consumir tokens infinitos
```

✅ Patrón Correcto

```
for step in range(max_steps): # ✅ Límite  
    estricto de seguridad
```



Consejo de Resiliencia en Producción

Guarda trazas completas de las decisiones del agente para auditoría y observabilidad.

Conclusiones Clave de la Sesión

Hemos cubierto los pilares teóricos y prácticos de **Clase 07: Agentes Autónomos y el Ciclo Cognitivo ReAct**. El dominio de esta unidad te proporciona una base sólida para afrontar la siguiente semana formativa.

Hitos Alcanzados

Comprensión profunda del modelo mental, capacidad de depuración de gotchas comunes y solidez en la sintaxis idiomática de Python 3.10+.

Notas de Repaso Rápido

Concepto	Punto Clave a Recordar
1. Modelo Mental	El Agente como un Detective que Piensa, Actúa y Observa hasta Resolver el Caso
2. Regla de Oro	Establece siempre un 'max_iterations = 5' para evitar que el agente quede atrapado en bucles infinitos.
3. Gotcha Crítico	Si una herramienta falla o devuelve error, el agente puede reintentar la misma acción en un ciclo sin fin.
4. Buenas Prácticas	Guarda trazas completas de las decisiones del agente para auditoría y observabilidad.

Mensaje del Instructor

¡Felicitaciones por completar esta lección! Recuerda que la constancia y el pedaleo diario en tu editor de código son los que te convertirán en un programador excepcional.

Fuentes Bibliográficas Oficiales

Para profundizar en los estándares formales del lenguaje y sus implementaciones internas, consulta la documentación canónica:

Recurso Oficial	Descripción	Enlace
Documentación Python 3	Especificación canónica y biblioteca estándar	docs.python.org/3/
PEP 8 — Style Guide	Estándar oficial de formateo y estilo	peps.python.org/pep-0008/
Real Python Tutorials	Patrones de desarrollo e ingeniería	realpython.com
Suite wisrovi en GitHub	Paquetes open source de alto rendimiento	github.com/wisrovi

🏆 Desafío Práctico para el Estudiante

🎯 Reto de la Semana

Añade una herramienta de calculadora matemática al agente y haz que resuelva una ecuación paso a paso.

🔧 Validación Automatizada

Ejecuta `pytest ejercicios/` en tu terminal de VS Code para verificar automáticamente la validez de tu solución.